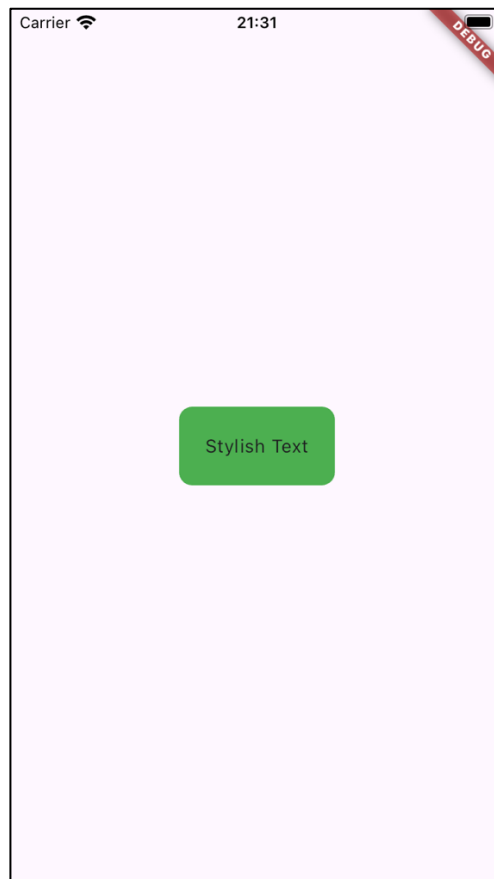




Why Does Flutter Use Functions and Properties Inside Widgets?

- a. Why Are Functions and Properties Used in Widgets?
 - Flutter allows functions and properties to be passed inside widgets.
 - This helps define both appearance (UI) and behavior (logic) in one place.
 - Instead of writing UI and logic separately (like older UI frameworks), Flutter keeps them together.
- b. Example: Adding Actions with onPressed (Button Example)
 - You can pass a function inside the onPressed property of a button widget.
 - This function tells the button what to do when clicked.
 - Both appearance (UI) and behavior (logic) stay together in one place.
 - This is example code :

```
ElevatedButton(  
  onPressed: () {  
    print('Button clicked!'); // This function runs when  
the button is pressed  
  },  
  child: Text('Click Me'), // The text shown inside the  
button  
);
```

- Explanation:
 - onPressed is a property where a function is passed.
 - The function executes when the button is clicked.

- The UI (Text('Click Me')) and logic (onPressed) are together.
- ◇ Separating Functions for Better Code Organization
 - Instead of defining functions inside widgets, you can define them separately and reference them.
 - This makes the code cleaner and easier to maintain.
 - Useful when the function is complex or used multiple times.
 - For example, this is code using separate function :

```
void buttonClicked() {
    print('Button clicked!'); // This function runs when
    the button is pressed
}

ElevatedButton(
    onPressed: buttonClicked, // Just reference the
    function by its name
    child: Text('Click Me'), // The text shown inside the
    button
);
```

- Benefits of Separating Functions:
 - ✓ *Reusability* → The same function can be used in multiple places.
 - ✓ *Clarity* → UI code remains clean and focused on design, while logic is handled separately.
 - ✓ *Maintainability* → If the button behavior changes, update only the function, not every button.
- c. Example: Using Properties for Styling (TextStyle in Text Widget)
 - Properties allow you to customize a widget's appearance.
 - Instead of defining styles separately, you can pass a TextStyle object inside the style property.
 - This makes customization more flexible and readable.
 - This is example code :

```
Text(
    'Hello, Flutter!',
    style: TextStyle( // TextStyle customizes the appearance
    of the text
        fontSize: 24, // Makes the text larger
        fontWeight: FontWeight.bold, // Makes the text bold
        color: Colors.blue, // Changes the text color to blue
    ),
);
```

Hello, Flutter!

- Explanation:
 - style is a property that takes a TextStyle object.

- `fontSize`, `fontWeight`, and `color` are keys that define text appearance.
- Customization is done directly inside the widget instead of a separate stylesheet.

Summary of Fundamental Concepts

- Key-value pairs in Flutter allow customization and flexibility.
- They override default values to modify UI appearance & behavior.
- Flutter's named parameters improve readability and maintainability.
- Flutter's key-value structure resembles JSON but serves different purposes.

- Nesting widgets allows structuring UI elements clearly.
- Keeps related components together, making code more readable.
- Customization is easier by wrapping widgets inside style-defining widgets.
- Flutter UI is entirely built using a nested widget tree.

- Functions inside widgets define behavior (e.g., `onPressed`).
- Separating functions improves reusability, clarity, and maintainability.
- Properties inside widgets customize appearance (e.g., `style` in `Text`).
- Flutter combines UI and logic in one place, making code easier to manage.